

# CSV → Kafka → Neo4j → Grounded AI Analytics Platform

Author: Tarun V • Status: Production Ready • Target: Docker Compose • Render • AWS S3 • Supabase

## 1. Executive Summary

**IntelliGuard** is an enterprise data streaming, graph database, and grounded AI analytics platform. It enables non-technical users to drag and drop previously unseen CSV datasets into a modern light-themed web interface. Every row is streamed asynchronously through **Apache Kafka**, ingested idempotently into **Neo4j Community Graph Database** via a high-throughput streaming loader, and queried by a **Dynamic Schema-Aware Grounded Chatbot**. Every answer is mathematically grounded in live graph nodes, returning the exact Cypher query and raw JSON results for 100% auditability with **zero hallucination**.

## 2. System Architecture & Infrastructure Stack

| Component        | Technology                   | Port / Location      | Functional Responsibility                                                                                     |
|------------------|------------------------------|----------------------|---------------------------------------------------------------------------------------------------------------|
| Web UI           | Vanilla HTML5/CSS/JS + Nginx | 3050 / Render \$PORT | Light-theme dashboard, drag-and-drop CSV zone, 20-row preview, live polling progress bar, grounded chat feed. |
| Backend API      | FastAPI (Python 3.11)        | 8001 / Cloud Web     | Handles /ingest, Kafka producer, schema introspection, Cypher translation, health checks, S3 & Supabase sync. |
| Message Broker   | Apache Kafka 3.7.0           | 9092 (KRaft Mode)    | Decouples upload burst speed from database writes; buffers row events with replayable topic partitions.       |
| Streaming Loader | Python Kafka Consumer        | Internal Worker      | Consumes row stream, executes idempotent MERGE into Neo4j, posts batch progress callbacks to API.             |
| Graph Database   | Neo4j 5.24 Community         | 7474 / 7687          | Stores :Dataset and :Row nodes with dynamic property sets and indexing for lightning-fast graph traversals.   |
| Relational DB    | Supabase (PostgreSQL 15)     | Cloud Database       | Persistent storage for datasets metadata, job progress across container restarts, and chat query audit logs.  |
| Object Storage   | AWS S3 Bucket                | Cloud S3 Storage     | Secure storage for raw uploaded CSV files under datasets/{dataset_id}/{filename} with pre-signed URLs.        |

## 3. Graph Data Model & Idempotency Guarantees

**Deterministic Idempotency Key:** Every file upload generates a stable 12-character ID via sha256(filename + content)[:12]. Each record receives a composite unique MERGE key {dataset\_id}:{row\_index}. Re-uploading the identical file or replaying Kafka produces **0 duplicate nodes**.

| Entity    | Properties Stored                              | MERGE Cypher Rule                                 |
|-----------|------------------------------------------------|---------------------------------------------------|
| :Dataset  | id, filename, uploaded_at                      | MERGE (d:Dataset {id: \$dataset_id})              |
| :Row      | key, dataset_id, row_index, ...all_csv_columns | MERGE (r:Row {key: \$key}) SET r += \$columns     |
| [HAS_ROW] | Directed relationship (d)-[:HAS_ROW]->(r)      | Guarantees strict dataset-to-row graph ownership. |

## 4. Dynamic Grounded Chatbot Engine

The chatbot implements dynamic schema introspection, token-to-column synonym resolution (e.g. *units* → quantity, *cost/dollars* → price, *buyer* → customer\_id), and multi-word graph value matching (e.g. Widget A, C7, pending). If a question references an entity not in the dataset (e.g. 'units in dollars'), the engine returns an honest refusal with grounded: false and lists the existing dataset columns.

## 5. Complete Acceptance Verification & Test Matrix

The system underwent thorough automated and manual acceptance tests across clean, large (10,000 rows), and hostile datasets:

| # | Question / Test Action | Synthesized Cypher Query                      | System Output / Answer                               | Status |
|---|------------------------|-----------------------------------------------|------------------------------------------------------|--------|
| 1 | Total Row Count        | MATCH (r:Row) RETURN count(r) AS total        | "There are 15 rows in the dataset."                  | PASS ✓ |
| 2 | Schema Discovery       | MATCH (r:Row) RETURN keys(r) LIMIT 1          | Lists all columns dynamically.                       | PASS ✓ |
| 3 | Entity Value Match     | MATCH (r:Row) WHERE r.product = 'Widget A'... | "There are 12.0 total units for Widget A."           | PASS ✓ |
| 4 | Aggregate Average      | MATCH (r:Row) RETURN avg(toFloat(r.price))    | "The average price is 37.32."                        | PASS ✓ |
| 5 | Category Grouping      | MATCH (r:Row) RETURN r.category, count(r)...  | Electronics: 8, Hardware: 5, Software: 2.            | PASS ✓ |
| 6 | Missing Entity Refusal | None (Ungrounded)                             | "Entity 'units dollars' was not found. Columns: ..." | PASS ✓ |
| 7 | Idempotency Replay     | MATCH (r:Row) RETURN count(r)                 | Count remains exactly 15 (0 duplicates).             | PASS ✓ |
| 8 | Hostile Empty CSV      | POST /ingest (0 bytes)                        | HTTP 400 Bad Request with error banner.              | PASS ✓ |
| 9 | Multi-Service Health   | GET /health                                   | {"status":"ok", "kafka":true, "neo4j":true}          | PASS ✓ |

## 6. Cloud Deployment & Cloud Database Configuration

**Render Cloud Deployment:** Configured via render.yaml Infrastructure Blueprint defining a Web Service running Dockerfile.render on dynamic \$PORT and a background Kafka worker.

**Supabase (PostgreSQL):** Dedicated SQL schema (supabase\_schema.sql) creating datasets, jobs, and chat\_logs tables with row-level security.

**AWS S3 Storage:** Uploaded CSV files are archived in S3 with pre-signed download URLs for long-term audit retention.

## 7. 1-Command Local & Cloud Reproduction Guide

```
# 1. Local 1-Command Startup
git clone git@github.com:tarunv307/IntelliGuard.git && cd IntelliGuard/csv-graph-chat
cp .env.example .env && docker compose up --build -d

# 2. Endpoints & Access
• Web UI Dashboard: http://localhost:3050
• REST API Docs: http://localhost:8001/docs
• Neo4j Browser: http://localhost:7474 (User: neo4j / Pass: csvgraphdb)

# 3. Render Blueprint Deployment
Go to dashboard.render.com → New + → Blueprint → Connect tarunv307/IntelliGuard
```

**Official Repository:** <https://github.com/tarunv307/IntelliGuard>